

MakenBrain — Document de suivi stratégique du projet

Mainteneur : Leonel MAKEN Dernière version connue du cerveau : 0.8.0 — Phase 2.8 "Trustworthy Brain" (21 juin 2026) Dépôt : github.com/leonelmaken/makenbrain Dernière mise à jour de ce document : 30 juin 2026

Ce document ne remplace pas ton CHANGELOG.md ni ton README.md, qui restent la source de vérité technique fichier par fichier. Il les complète en suivant la **trajectoire stratégique** : où en est le cerveau dans sa transformation vers un système capable de s'auto-améliorer et d'assister des décisions à fort enjeu, qui valide quoi, et quels risques sont sous contrôle.

1. Vision

MakenBrain doit devenir un cerveau numérique personnel capable de :

- raisonner avec mémoire, graphe de connaissances et logique, en signalant honnêtement son incertitude (déjà acquis en Phase 2.8) ;
- observer son propre fonctionnement et se proposer des améliorations**, que Leonel valide avant toute implémentation — pour ne plus dépendre uniquement des sessions avec Claude ;
- assister des décisions à enjeux élevés (immigration, embauche, investissement, détection d'arnaque, prédiction) avec une traçabilité totale ;
- accueillir, sous contrôle d'un Super Admin, un nombre croissant d'utilisateurs sans saturer le calcul ni le stockage ;
- devenir, à terme, le cerveau central reliant tous les projets de Leonel (MakenBrain, SmartBudget Africa, INTIA, candidatures Canada).

2. Carte des deux pistes existantes

Le code actuel avance sur deux pistes parallèles. Les nouvelles phases (section 3) se construisent **au-dessus** des deux, pas à côté.

Piste	Contenu	Statut
A — Capacités (mi-juin)	Fondations FastAPI/Ollama/ChromaDB/NetworkX, agent fichiers (propose→valide→agit, sandbox, backups, diff, audit), domain explorer, scheduler autonome (recherche + rapport quotidien), personnalité, conscience, serveur MCP, génération image/vidéo, routage Ollama/Groq/Gemini	✅ Acquis
B — Sécurité & Confiance (depuis le 20 juin)	Phase 1 Secure Core (auth, CORS, rate limiting, sandbox stricte) ; Phase 2 User Brain (Supabase Auth, mémoire par user_id) ; Phase 2.8 Trustworthy Brain (confidence, risk_level, self_critic, suggested_sources)	✅ Acquis jusqu'à 2.8
B — suite	Phase 3 Production Hardening (observabilité, rôles fins, rotation des secrets, UI admin sécurité) ; Phase 4 Assistant Personnel Étendu (emails, calendrier, comptes externes)	📅 Planifié dans ton propre README, pas encore commencé

3. Feuille de route consolidée (vue d'ensemble)

#	Phase	Statut	S'appuie sur
1	Fondations techniques	✔ Fait	—
2	Agent fichiers & sandbox	✔ Fait	—
3	Exploration & veille autonome	✔ Fait	—
4	Personnalité, conscience & MCP	✔ Fait	—
5	Orchestration multi-fournisseurs	✔ Fait	—
6	Sécurité & cerveau de confiance (v0.8.0)	✔ Fait	Phases 1-5
7	Renforcement production	📅 Planifié (ton README, Phase 3)	Phase 6
8	Auto-évolution contrôlée	🔄 En cours de conception	proposals.json, scheduler.py, pattern agent.py
9	Décisions à enjeux élevés	📅 Planifié	response_confidence.py, self_critic.py, source_router.py
10	Super Admin & bêta	📅 Planifié	UserService, Supabase Auth
11	Gestion des ressources	📅 Planifié	providers.py, audit.py
12	Assistant personnel étendu	📅 Planifié (ton README, Phase 4)	Phase 11
13	Écosystème MakenBrain	🌐 Vision long terme	Tout ce qui précède

Les phases en gras (8 à 11) sont celles que tu m'as demandées aujourd'hui. Le détail de chacune suit, avec les fichiers réels de ton dépôt à réutiliser ou créer.

4. Phase 8 — Auto-évolution contrôlée

Objectif : que MakenBrain analyse son propre usage, te propose des améliorations, et — une fois que tu as validé — les implémente lui-même dans un environnement isolé, sans jamais déployer seul sur main.

Ce qui existe déjà et qu'il ne faut pas reconstruire :

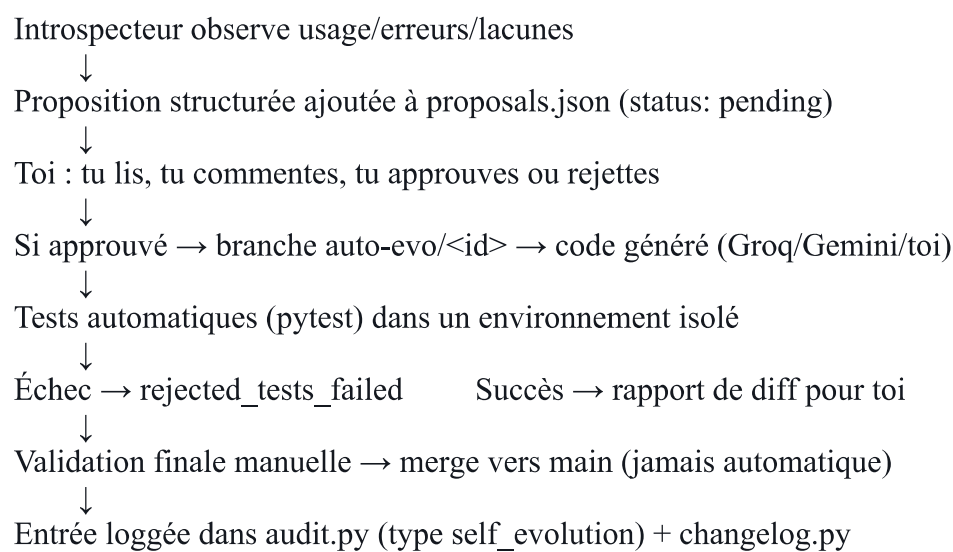
- brain_data/proposals.json a déjà le bon schéma (id, title, description, action_type, risks, status: pending/approved/done). Il scanne aujourd'hui des projets externes (ex. SmartBudget Africa) — il faut juste lui ajouter une cible interne.
- core/scheduler.py génère déjà un rapport quotidien avec une section "💡 *Ce que je suggère à MAKEN de travailler*". C'est exactement la graine d'un introspecteur — il manque juste la transformation de ce texte en propositions structurées.
- Le pattern dry_run + apply_changes + create_backup + unified_diff + audit_event de routers/agent.py (/agent/write, /agent/modify) est déjà robuste. Réutilise-le tel quel, ne le réinvente pas.
- **Point d'attention central :** core/sandbox.py interdit explicitement les chemins hors workspace/, projects/, uploads/ — donc core/, routers/ et models/ (le code de MakenBrain lui-même) sont aujourd'hui **inaccessibles** à l'agent. C'est une bonne protection à garder, mais cela veut dire qu'il faut un sandbox dédié, séparé et plus strict, pour l'auto-évolution.

À construire :

- ☐ core/self_sandbox.py — sandbox dédié limité à core/, routers/, models/, avec une liste blanche explicite de fichiers modifiables et une liste noire renforcée pour les fichiers sensibles (auth.py, supabase_client.py, config.py, version.py) exigeant une double validation.
- ☐ core/introspector.py — reprend la logique du rapport quotidien de scheduler.py, mais écrit des entrées structurées dans proposals.json avec un nouveau champ target: "makenbrain" | "external_project".

- ☐ Endpoint `/agent/self-evolve/propositions` — liste les propositions en attente concernant MakenBrain lui-même, avec les mêmes garanties `dry_run/apply_changes` que `/agent/write`.
- ☐ Étape de tests obligatoire avant tout `apply_changes=true` sur le code de MakenBrain : exécution automatique de `pytest tests/` dans un environnement isolé ; en cas d'échec, la proposition passe en status: `rejected_tests_failed` et n'est jamais appliquée.
- ☐ Branche Git dédiée par proposition validée (`auto-evo/<id-proposition>`), jamais de commit direct sur `main`.
- ☐ Quota de sécurité configurable dans `scheduler_config.json` (ex. `max_self_proposals_per_week`) pour éviter l'emballement.
- ☐ Nouveau type d'événement `self_evolution` dans `core/audit.py`, tracé séparément des audits classiques.
- ☐ Interface de validation : commence simple via Swagger (`/agent/self-evolve/approve, /reject`), une vraie UI viendra avec la Phase 10 (Super Admin).

Boucle complète visée :



5. Phase 9 — Décisions à enjeux élevés

Objectif : immigration, embauche, investissement/trading, détection d'arnaque, prédiction — avec une rigueur adaptée à l'enjeu réel de chaque sujet.

Bonne nouvelle : ta Phase 2.8 "Trustworthy Brain" (`response_confidence.py`, `self_critic.py`, `source_router.py`) est *exactement* l'infrastructure de fiabilité dont ces modules ont besoin. Tu n'as pas de nouvelle fondation à inventer — chaque module de décision doit simplement s'y brancher.

À construire :

- ☐ Nouveau package `core/decision_modules/` avec un fichier par domaine : `immigration.py`, `embauche.py`, `investissement.py`, `anti_arnaque.py`, `prediction.py`.
- ☐ Chaque module appelle `response_confidence.py` pour le score, `self_critic.py` pour la prudence automatique si `confidence < 0.60`, et `source_router.py` pour proposer des vérifications externes.
- ☐ Champ `disclaimer` obligatoire et spécifique par domaine, renvoyé dans chaque réponse (voir tableau ci-dessous).
- ☐ Extension de `core/audit.py` : chaque recommandation à enjeu élevé crée une entrée immuable (domaine, score de confiance, sources citées, horodatage) — indispensable si ces modules servent un jour à de vraies décisions.
- ☐ **Ordre de construction recommandé :** commence par `anti_arnaque.py` (le moins réglementé, le plus simple à valider), puis seulement ensuite les domaines plus sensibles.

Domaine	Garde-fou spécifique	Positionnement du module
Anti-arnaque	Croisement de bases de fraude connues + signaux linguistiques	Outil de vérification, premier module à livrer
Embauche	Audit automatique de biais (âge, genre, origine) avant d'exposer une recommandation	Aide à la présélection, jamais décision finale automatique
Immigration	Disclaimer obligatoire : "n'est pas un avis juridique"	Assistant de recherche et de structuration de dossier
Investissement / trading	Toujours une fourchette ou une probabilité, jamais une certitude binaire	Outil d'analyse de données, pas un conseil en investissement réglementé
Prédiction	Intervalle de confiance et méthode toujours affichés	Estimation probabiliste, jamais une affirmation catégorique

Pour rappel : je ne suis ni avocat ni conseiller financier, et le même principe doit s'appliquer à MakenBrain lui-même une fois ces modules en place — il doit se présenter comme un outil d'aide à la décision, pas comme l'autorité finale, surtout pour l'immigration et l'investissement où la réglementation locale varie beaucoup.

6. Phase 10 — Super Admin & bêta multi-utilisateurs

Constat actuel : ton README liste toi-même "gestion fine des rôles" comme manquant. Aujourd'hui, `core/permissions.py` ne gère que des chemins de fichiers, pas des rôles utilisateurs — il n'y a qu'un bearer token Supabase, sans distinction Super Admin / Admin / Bêta / Utilisateur.

À construire :

- ☐ Ajouter un champ `role` à `models/user.py` (`super_admin`, `admin`, `beta`, `user`) — absent aujourd'hui.
- ☐ `core/rbac.py` — contrôle d'accès basé sur les rôles, au-dessus de `UserService` existant.
- ☐ Étendre `routers/users.py` : validation manuelle de compte, suspension, bannissement, vue d'ensemble des utilisateurs et de leur usage.
- ☐ Flux d'inscription : Inscription → email → validation manuelle (toi, Super Admin) → compte actif → connexion autorisée — automatisable plus tard.
- ☐ Quotas par rôle bêta : requêtes/jour, accès en lecture seule (au début) aux modules de la Phase 9.
- ☐ **Kill-switch** : un Super Admin doit pouvoir couper instantanément l'accès d'un utilisateur ou d'un module entier, sans redéploiement.
- ☐ Tableau de bord minimal (Swagger suffit au début) : utilisateurs actifs, requêtes récentes, coûts cumulés par fournisseur.

7. Phase 11 — Gestion des ressources (calcul, stockage, coûts)

Constat actuel : `core/providers.py` (`detect_provider`) bascule vers Groq dès que la clé est configurée, sans réel calcul de coût ou de charge. L'observabilité production est déjà notée "priorité haute, manquante" dans ton propre README.

À construire :

- ☐ Transformer le routage en politique à plusieurs niveaux : Ollama local pour les tâches courtes/privées, Groq pour la majorité des tâches, Gemini réservé au contexte long — avec un compteur d'usage par fournisseur.
- ☐ `core/usage_monitor.py` — agrège requêtes/jour par fournisseur, tokens consommés, coût estimé, taille de `brain_data/` (ChromaDB + JSON), avec alertes à 70 % et 90 % d'un seuil mensuel que tu définis.
- ☐ Politique de rétention de la mémoire vectorielle : score d'importance + archivage à froid des souvenirs anciens peu consultés, plutôt que suppression ou croissance infinie.
- ☐ Cache des réponses/embeddings pour les requêtes répétées ou très proches.
- ☐ Vérifier que `core/rate_limit.py` couvre aussi la charge agrégée multi-utilisateurs, pas seulement par endpoint individuel — important dès l'ouverture de la bêta (Phase 10).

- ☐ S'assurer que les sauvegardes de `core/backups.py` restent des deltas et non des copies complètes, pour ne pas saturer le disque quand le nombre d'utilisateurs augmente.

8. Phases 12-13 — Vision plus loin

- Phase 12 (déjà dans ton README, "Phase 4") :** emails, calendrier, comptes externes, automatisation avancée, vérification externe automatique des sources.
- Phase 13 — Écosystème MakenBrain (vision long terme) :**
 - Étendre `scheduler_config.json` (`domains_to_watch`) avec des sujets liés à tes autres priorités (immigration Canada, droit du travail Cameroun, fintech Afrique déjà présent).
 - "Second cerveau" unifié : relier `core/graph.py` à tes autres projets (SmartBudget Africa, INTIA) dans un graphe de connaissances commun.
 - Mode "simulation de décision" pour tes grandes décisions personnelles, en s'appuyant sur les modules de la Phase 9.
 - Marketplace d'agents tiers, version mobile/offline allégée, organisation multi-agents (CEO/CTO/Backend/Frontend) — en s'appuyant sur `core/providers.py` plutôt qu'en recommençant de zéro.

9. Indicateurs à suivre

Indicateur	Où le mesurer	Seuil d'alerte suggéré
Requêtes/jour par fournisseur	<code>usage_monitor.py</code> (à créer)	—
Coût cumulé mensuel (Groq/Gemini)	<code>usage_monitor.py</code>	70 % puis 90 % du budget
Taille de <code>brain_data/</code> (Mo)	disque	80 % de l'espace alloué
Score de confiance moyen des réponses	<code>response_confidence.py</code>	< 0.6 fréquent = lacune de connaissance
Propositions <code>self_improvement</code> en attente	<code>proposals.json</code>	> 10 en attente = revoir le quota
Taux d'acceptation des propositions	<code>proposals.json</code> (champ <code>status</code>)	< 30 % = recalibrer l'introspecteur
Utilisateurs bêta actifs / invités	<code>UserService</code>	—
Incidents de violation de sandbox	<code>audit.py</code>	tout incident = à examiner sous 24h

10. Registre des risques & conformité

Domaine	Risque	Mitigation
Auto-évolution	Modification d'un fichier sensible (<code>auth.py</code> , <code>config.py</code>)	<code>self_sandbox</code> avec liste blanche stricte + double validation
Auto-évolution	Boucle de propositions incontrôlée	Quota hebdomadaire + validation humaine systématique
Décision — embauche	Biais discriminatoire dans une recommandation	Audit de biais automatique avant exposition
Décision — immigration	Confusion avec un véritable avis juridique	Disclaimer obligatoire + positionnement "aide à la recherche"
Décision — investissement	Confusion avec un conseil financier réglementé	Disclaimer + jamais de certitude binaire
Bêta multi-utilisateurs	Un testeur épuise les ressources partagées	Quotas par rôle + kill-switch
Stockage	ChromaDB grossit indéfiniment	Politique de rétention + archivage à froid

11. Journal d'évolution

Date	Version	Type	Description	Statut
2026-06-20	0.6.0	Sécurité	Phase 1 Secure Core (sandbox, audit, backups, rate limiting)	 Fait
2026-06-21	0.8.0	Confiance	Phase 2.8 Trustworthy Brain (confidence score, self-critic, source router)	 Fait

(Ajoute une ligne à chaque release — ce tableau devient ta mémoire de projet la plus fiable.)

12. Glossaire rapide

- **MCP** — Model Context Protocol, la couche qui expose les capacités du cerveau à d'autres outils.
- **Sandbox** — répertoire strictement délimité dans lequel l'agent peut lire/écrire ; tout le reste est interdit par défaut.
- **Dry-run** — simulation d'une action (diff généré) sans écriture réelle, avant confirmation explicite.
- **Self-critic** — module qui ajoute une formulation prudente quand la confiance est faible.
- **Confidence score** — score 0.0 → 1.0 qui évalue la fiabilité d'une réponse avant de l'exposer.
- **RBAC** — Role-Based Access Control, contrôle d'accès basé sur les rôles (à construire en Phase 10).
- **Introspecteur** — module qui analyse l'usage du cerveau pour en déduire des pistes d'amélioration.

13. Prochaines actions immédiates

1. Lancer manuellement les tests Phase 2.8 (`pytest tests/test_response_confidence.py`, etc.) — déjà noté "à faire" dans ton propre README, avant d'empiler de nouvelles phases sur du code non vérifié en pratique.
2. Créer `core/self_sandbox.py` en copiant la structure de `core/sandbox.py`, avec une racine différente et une liste blanche de fichiers.
3. Ajouter manuellement un premier exemple de proposition `action_type`: "self_improvement" dans `proposals.json`, pour valider le schéma avant d'automatiser l'introspecteur.
4. Ajouter le champ `role` à `models/user.py` et créer ton propre compte Super Admin avant d'envisager le moindre bêta-testeur externe.
5. Créer `core/usage_monitor.py` minimal qui logge juste les requêtes/jour par fournisseur — version la plus simple possible, à enrichir ensuite.